

System and method for hyperspectral artificial vision for machines

Problem

We want to create an artificial vision which is analogous to human vision - in general analogues to biotic vision. The biotic vision uses natural light reflected from the object to extract the spatial, temporal, and chemical properties of the object. Example spatial properties are objects geometric properties such as size and shape, surface texture and so on. Example of temporal properties are objects motion. Example of chemical properties are chemical composition of object or material of the object. The last property is – in common sense – color. Color is a psychological phenomenon which associates certain mental sensation (which commonly called as a color) with the energy reflected by the object. This is by tapping into the energy matter interaction at the molecular level. Depending upon the material and its chemical composition, energy of certain wavelengths is absorbed by the material and the rest is reflected, (some part is scattered away from the direction of the viewer). For example, ripe mango would appear red whereas, young would appear green, a dry green leaf appears yellow as they reflect both red and green, whereas a fresh leaf appears green because red is absorbed by chlorophyll in the leaf. Thus “color” is a discriminator designed for separating objects/or detecting objects. Especially indicating chemical composition.

Following are the elements of biotic vision: optical components, sensors (rods and cones), perceptual computation that is color (by layers of natural neural network). We will skip the optical part here as it is not in the current scope. Sensors determine how they respond to light. Humans have three cones responding to short wavelengths (~400 nm to ~500 nm), medium wavelengths (~500 nm to ~600) nm and longer wavelengths (~600 nm to ~700 nm) in visible spectrum. They are corresponding to the blue, green, and red sensation. All the colors we see are the result of excitations of these cones in some degree. These are not exclusive regions as the response functions peak at certain wavelengths and the limbs of response function overlap each other. Thus, any color is a sensation resulted combination of excitement of three types of cones. For example, 100% excitation of red and green would result in yellow color sensation. Design of sensors and neural processing thus includes – deciding -

- Number of sensors or primitives, their spectral response function (excitation vs wavelength), and the positions for peak and the distance between peaks (overlap of 2 SRFS)
- Neural connection to process the signal

We learn the number of primitives and their SRFs and their overlap that are optimal in each situation

This is inspired by nature, where certain animals are more primitive than humans, which have evolved to help them better perceive their surroundings (n=3 for humans, 4 for birds, and 16 for mantis shrimps).

The adequate number of primitives varies depending on the set of materials to be seen by machine, objective here is to determine the appropriate number as well as their SRFs. Color is being used as a discriminator in this case, which can aid in locating the target material.

Other choices are – one can fix the number of primitives to a particular number – say three. The same is true for SRF – they are generally fixed in case of RGB machine vision or robot vision, even in case the sensing goes beyond 800 nm.

To determine the optimal numbers (primitives) is hard or practically impossible as this entails changing sensors or filters dynamically during a robot/machine operation. Even if we use tunable filters and apply different voltages to them or rotate/move grating system or move object lense for dispersing different wavelength of light and capture it by CCD later, the images for every wavelength would have some time lag and hence may not work in dynamic situation (this is same as recording a single wavelength band at a time – so let's imagine we have 100 bands then 100th band would be captured after $t+(100 \times p_t)$ where p_t is processing time for a single band).

We use artificial neural network and its specific design to learn the SRF of a primitive, and the number of primitives as well

Trial and error method – we can begin with CIE sensitivity curves or simple triangular SRFs. We can add the perturbations in the SRFs and then perform the task. Based on the outcome we can update the perturbations and correct or improve the outcome. This process can be performed iteratively to come up with the SRFs that are optimal. Same can be done for overlap decision. As can be seen this process is complex and may not result in optimal SRFs. Overall time complexity of these types of methods would be very large.

Our method deploys filter layer in such a way that it learns SRF automatically (details in respective section). The learned model with optimal SRF and number of primitives in a given task are uploaded on neuromorphic chip of machine that performs vision tasks. The SRF can be learned and updated as per the task dynamically as well.

Components of machine and the workflow

hyperspectral sensor/camera, artificial vision module (training and deployment), controller, motors,

For a given set of materials, the task is to find the appropriate number of filters and spectral response functions (SRF). This is inspired by nature, where certain animals have more primitive than humans, which have evolved to help them better perceive their surroundings (n=3 for humans, 4 for birds, and 16 for mantis shrimps).

The present invention discloses a system and method for developing n-chromatic vision for detecting and responding to a target material.

(If it is a fire, extinguish it; if it is a fruit, pick it up; if it is a hazardous chemical raise alarm; if it is an oil on road, then adjust the traction by increasing down force and rpm of the car Tyre).

The technical invention is to

Enable the machine to see (and discriminate) using more than three chromatic primitives

bring n-chromatic vision capabilities to machine vision by dynamically determining the appropriate number of primitives based on materials.

The adequate number of primitives varies depending on the set of materials, objective here is to determine the appropriate number as well as their SRFs. Color is being used as a discriminator in this case, which can aid in locating the target material.

The color changes based on the number of primitives and their SRFs. The machine enabled with such an optimal vision responds to the visual stimulus in a better manner, locates the object more precisely and interact with the surroundings in a better manner. Not only that the chromatic primaries determined are optimal for discriminating materials within the given scene at a given time.

A simple example, after locating the target the machine sends a signal to actuate the peripherals to grab the object. This is equivalent to adding optimal number of cones (with unique response) dynamically as per the scene.

Solution Architecture

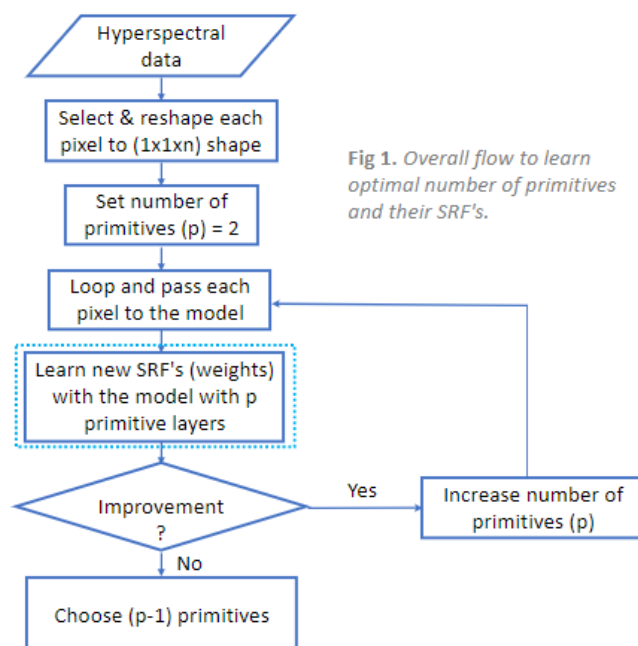


Fig 1. Overall flow to learn optimal number of primitives and their SRF's.

A hyperspectral data cube to obtain spectral signatures and its ground truth file is being used to get signature labels.

The input to the neural network is a single pixel which has information across all wavelengths. The input is reshaped to $1 * 1 * n$ where n denotes the number of bands in the hyperspectral image.

Our aim is to discover the best number of primitives (p), and we start with $p = 2$. We iteratively cycle through all training data (pixels), updating weights for each unsuccessful material class prediction. Model with two primitives serves as our baseline, after which we add another primitive layer and repeat the training procedure.

After training is completed, we compare it to see if there is any significant improvement in the new model over the previous model. If there is growth, we will continue to add more primitive layers. We keep repeating these processes until we obtain convergence, i.e., the new model does not improve. This is where we come to a halt, and we obtain the optimal number of primitives for the given materials ($p-1$). For example, for the Pavia Centre dataset, we began with two primitives. As we saw improvements, we continued to add more primitive layers. However, the model with $p = 5$ primitives did not show any significant improvement over the model with $p = 4$, so we stop here and get the optimal number as ($p-1 = 4$) and SRF's for those four primitives.

Training

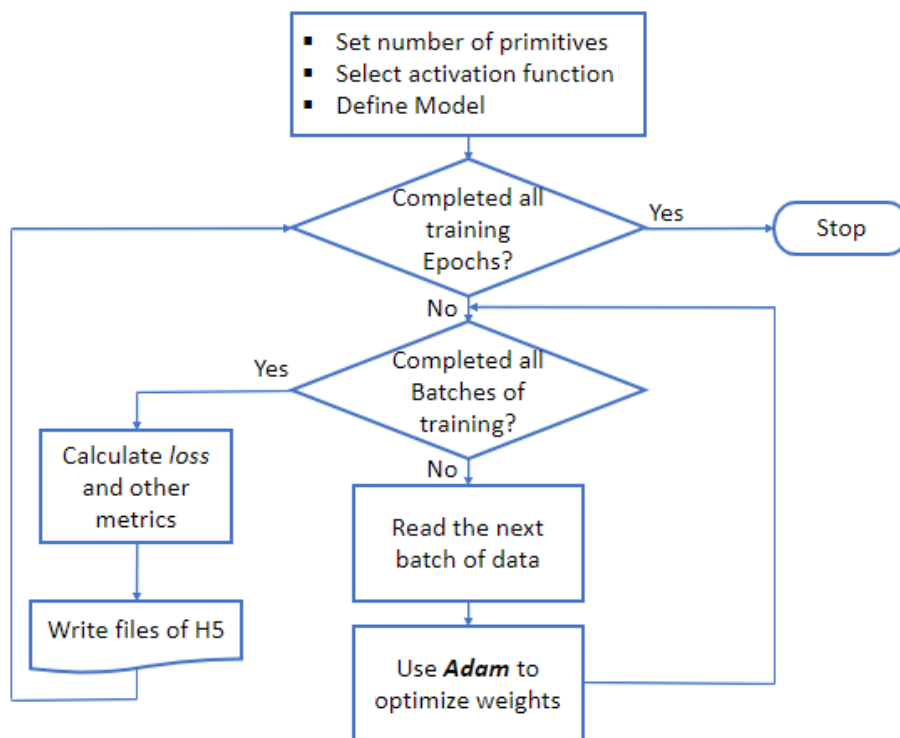


Fig.2. Neural Network Training

We begin by selecting the number of primitive layers to be used (This will be done dynamically in the outer flow where we keep adding more primitives as explained above). As an activation function, we choose either "relu" or "sigmoid". These hyperparameters are used to define the model. We loop for n epochs. Data is divided into batches for each epoch. We select "adam" as an optimizer. Weights are

updated after each batch. Once all of the batches have been used for training, we compute the loss and metrics specified during model compilation. These metrics and weights are saved, and the model is trained for another epoch.

Neural Network

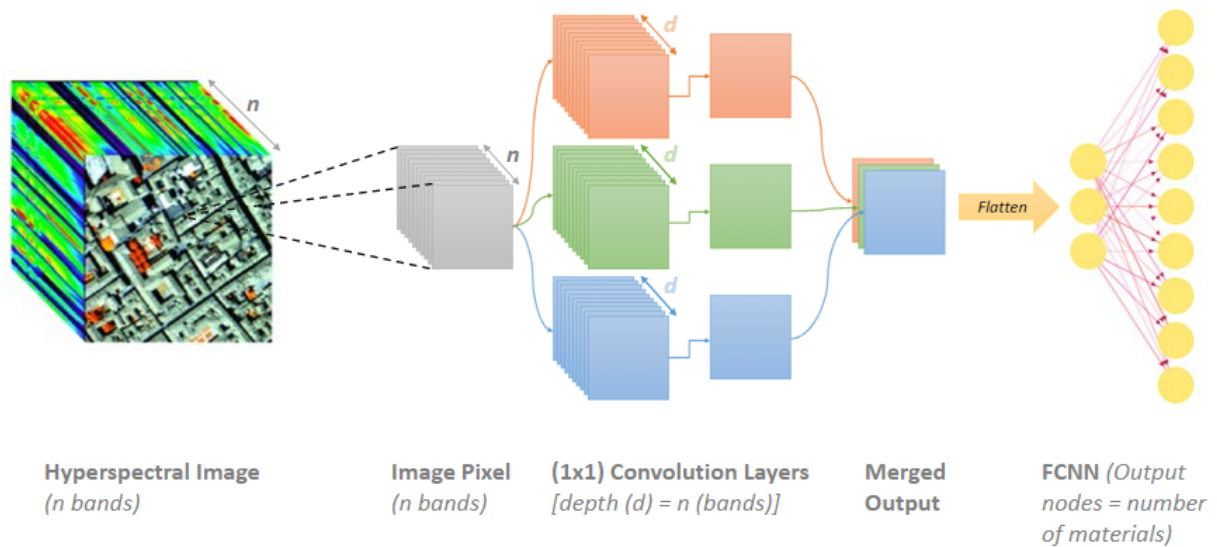


Fig.3. Neural Network Architecture

The figure above illustrates a hyperspectral data cube with n bands, followed by neural network architecture. For training, we use a single pixel at a time, which is represented by grey squares. This input signature is sent to p primitive layers (here 3). These are 1×1 Convolution layers with a depth of d , where d and n are both equal. Each of these layers produces one output. To get new values, we stack them. These results are then fed into a fully connected neural network, which predicts their class.

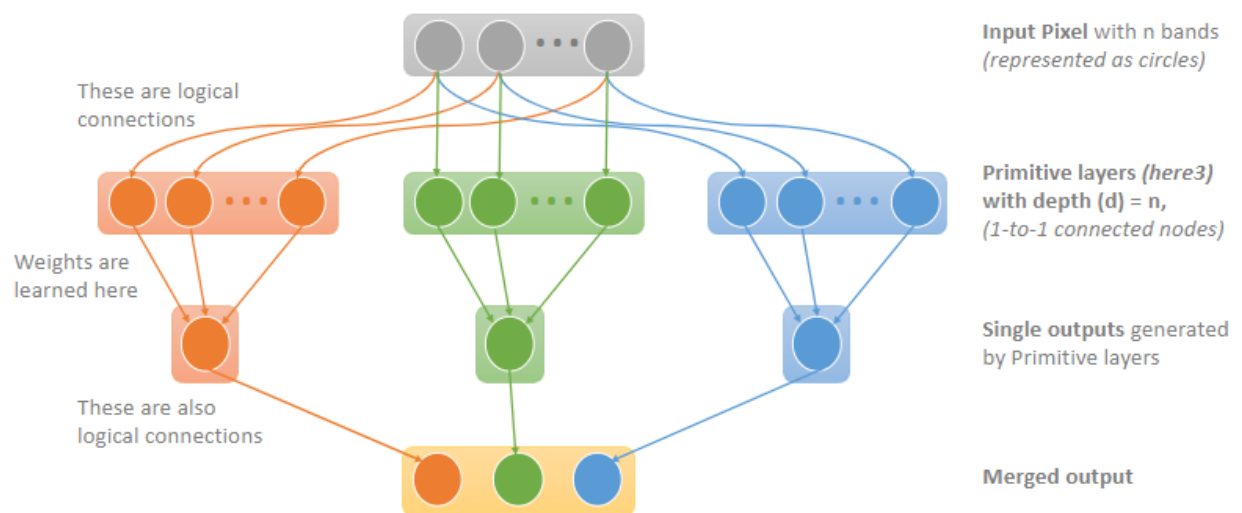


Fig.4. Internal working of convolution layers

The diagram above depicts a thorough perspective of neural network architecture. As an input, the spectral signatures of n bands are used (grey box). These layers are linked to parallel primitive layers of depth d . The architecture presented above is made up of three primitives. The architecture begins with three primitive layers, and the number of layers increases as it converges to an optimal number. Each input node is linked to a single primitive layer node. These are logical connections, and weights are not learned here. These primitive layers, which are made up of 1×1 CNN layers, produce a single node output. So, we get a connection to a single node from n nodes. This is where the model learns weights. Weights learned by these primitive layers (1×1 convolution layers) are interpreted as spectral response functions SRFs. We create an SRF for each primitive layer. These single pixel outputs are now stacked to generate new color pixel values. Using pretrained SRF values, these color values can be used to generate images.

Deployment

These Spectral Response Functions (SRFs) can then be transferred to an online deployable system to detect target material. To better complement fresh target content, these SRFs can be updated online, or a new Reinforcement Learning approach can be applied.

Type of data as input for deployment:

- Upload of HS Data post capturing.
- Real-Time hyperspectral Data captured through HS sensors

The SRFs can be deployed to an embedded machine which has to perform target detection, this type of deployment is done when the machines scan for different types of materials which are already known to the model as the model has already learned SRFs for these materials.

The SRFs can also be deployed in real time systems where the target to be detected is unknown, in this scenario the model learns new SRFs dynamically by finding the optimal n for a particular combination of target and data.

Experiments

- ***Experiment to learn Tri Chromatic color vision for visible range bands:***

Experiments 1 and 2: Experiment 1 was to design a neural network architecture capable of learning color filters similar to human trichromatic vision. The architecture was designed with three parallel 1×1 convolution layers to learn the filters. For this experiment, we used data from the visible region (400nm to 700nm) of the spectral signature from the Pavia University dataset. This data was used as an input, and the material label was obtained from the ground truth file. Filters learned from the model (Fig 6) had similarities to human vision but also distinct. Experiment 2 involved using CIE filters (Fig 5) as model weights and comparing the results to Experiment 1. It was discovered that Learned filters outperformed the model using CIE filters, implying that CIE weights may not be ideal.

We also discovered that each learned primitive was one to one related with one of the CIE filters when we used Spectral Angle Mapper (SAM). The table below contains detailed values. Each value in the

table shows the angle (in degrees) between two vectors, and the lower the angle, the more closely the two curves are represented.

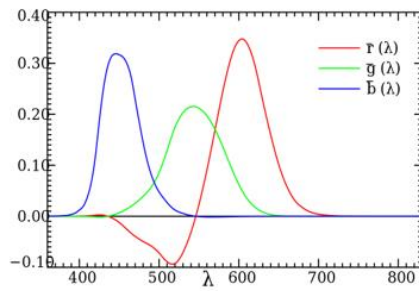


Fig.5. CIE XYZ

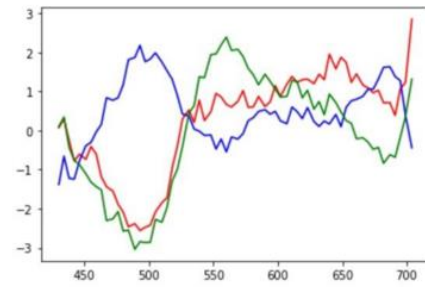


Fig.6. Learned SRF

Experiment	Number of Primitives	Input Shape	Weight Initialization	Number of epochs	Accuracy
1	3	(1, 1, 66)	Random	30	76%
2	3	(1, 1, 66)	CIE	30	70%

SAM similarity between Fig.5 & Fig.6 (values in degrees)	Learned primitive 1	Learned primitive 2	Learned primitive 3
CIE red filter	1.43989279225721	1.60557333649109	1.67997308748719
CIE green filter	1.54431323123107	1.10721981846080	1.90183977712053
CIE blue filter	1.74716932415535	1.41876129725418	1.35260928922169

- Experiment to learn Tri Chromatic color vision for available bands**

Experiment 3: We chose the complete spectrum of the material as an input to the model in this experiment. This was done to check if the additional information affected the accuracy of the models. SRFs that had been learned were then utilized to generate a new image (Fig 8.).



Fig.7. RGB composite



Fig.8. Generated Image

Experiment	Number of Primitives	Input Shape	Weight Initialization	Number of epochs	Accuracy
3	3	(1, 1, 103)	Random	35	76%=77%

- **Experiment to learn n – chromatic color vision for available bands**

Experiment 4: Experiment 4 entailed testing with various numbers of primitive levels (1x1 Convolution layers). Experiments were conducted using 4, 7, 10, 15, and 21 primitives. **The accuracy of these models improved as the number of primitives increased**, but there was no noticeable gain once we crossed the 7 primitive mark. **These results indicated that just 3 primitives were not ideal for the given set of materials.**

Experiment	Number of Primitives	Input Shape	Weight Initialization	Number of epochs	Accuracy
4	4	(1, 1, 103)	Random	40	80%
	7	(1, 1, 103)	Random	50	86%
	10	(1, 1, 103)	Random	55	88%
	15	(1, 1, 103)	Random	55	90%
	21	(1, 1, 103)	Random	60	91%

Again, SAM was used to discover similarities between CIE curves and learning primitives from the model, which had four primitives.

SAM	Primitive 1	Primitive 2	Primitive 3	Primitive 4
CIE red filter	1.21709753288	1.69147905722	1.57256632771	1.94628801025
CIE green filter	1.76669693923	1.70456491330	1.27526310136	2.52662134444
CIE blue filter	1.41551302351	0.49223247543	1.03319168395	1.37561950759

According to the above table, three of the primitives were one to one coupled to a CIE filter, and the fourth primitive had varied values.

- **Experiment to learn n – chromatic color vision for available bands with shadow class removed**

Experiment 5: The above experiments were repeated after the shadow class was removed from the training data. This was done to determine if the model could accurately identify the underlying material in the image's shadow region. The outcomes are shown in (Fig. 10).

- **Experiment to learn n – chromatic color vision for available bands with Pavia Centre Dataset**

Experiment 6: We did identical experiments using the Pavia Centre dataset, varying the number of primitives, i.e., 3, 4, 7, and 10 primitive layers. This dataset contained a different set of materials. Our purpose was to examine how changes in materials affect SRFs. In these tests, the model reached its peak accuracy with only 4 primitives, and adding more primitives did not increase accuracy any more.

Experiment	Number of Primitives	Input Shape	Weight Initialization	Number of epochs	Accuracy
6	3	(1, 1, 65)	CIE	20	92%-96%
	3	(1, 1, 65)	Random	20	94%-95%
	3	(1, 1, 102)	Random	20	94%-96%
	4	(1, 1, 102)	Random	20	97%
	7	(1, 1, 102)	Random	20	97%-98%

- **Experiments with Indian Pines Dataset**

Experiment 7: Experiments were repeated on this dataset to strengthen the hypothesis. However, the classification accuracy was between 50 and 55 percent. When compared to the original RGB image, the generated image had improved properties. The presence of similar classes in the dataset resulted in lower accuracy. Because most of the classes were vegetation, it was even more difficult to tell them apart.

- **Experiments with Urban Dataset**

Experiment 8: Materials in the urban dataset include asphalt, grass, trees, roofs, metal, and dirt. With only four primitives, the model was able to achieve 95% accuracy.

Results

- With the preceding studies, it was clear that learned SRFs performed considerably better than human vision for a given scene/set of materials, and that more primitives may be necessary to better identify target material. The number of primitives varies based on the materials, as do the SRFs for those primitives.
- When we compare the original image (Fig 7.) to the generated image, we can clearly discern between different materials based on color. In addition, the generated image is brighter, sharper, and more saturated than the RGB image. These visual gains are also backed up by numbers, as shown in the table (Fig 9.).

Data	Metrics				
Pavia University	Contrast	Sharpness	Brightness	Saturation	Hue
RGB Image	1.0	7.799	0.172	109.870	34.344
Generated Image	0.839	12.296	0.454	217.468	119.445

Data	Metrics				
Pavia Centre	Contrast	Sharpness	Brightness	Saturation	Hue
RGB Image	1.0	9.942	0.130	120.608	36.584
Generated Image	0.841	14.031	0.401	207.817	101.280

Fig.9. Image property comparison

- It was also clear from the generated photos that the model could accurately identify the underlying material, which seemed black due to shadows in original RGB image (Fig 10.).



Fig.10. Materials identified in shadow region (Experiment 5)